En este apartado, seguimos una dinámica similar a la del apartado 2, ya que nuevamente tenemos que entrenar un modelo para realizar predicciones. Además, hemos experimentado con tres técnicas diferentes para lograrlo.

- **Método basado en Lexicons**

En primer lugar, implementamos un método basado en *lexicons*. En este enfoque, utilizamos un léxico llamado *VADER*, que es un modelo preentrenado para el análisis de sentimientos. Este modelo devuelve puntuaciones de sentimiento en forma de un diccionario con valores que representan aspectos como la positividad, la negatividad y el puntaje general (obtenido sumando los valores de positividad y negatividad para determinar si el resultado general es positivo o negativo). *VADER* nos ofrece una gran cantidad de información, a diferencia de otros conjuntos de datos; lo descubrimos mediante investigación y vimos que se puede implementar fácilmente con una librería en Python. A continuación, creamos la función *score_sentence_vader*, que coge una oración y devuelve una etiqueta de sentimiento "POSITIVE" si el puntaje compuesto (la medida global del sentimiento) es mayor que 0, o "NEGATIVE" si es menor o igual a 0.

Es importante aclarar que el método *analyzer.polarity_scores(sentence)* devuelve un diccionario con varias puntuaciones de sentimiento, donde la clave *compound* es la que representa el puntaje global del sentimiento (positivo o negativo).

- **Método basado en palabras únicas**

En cuanto al segundo método, se basa en crear vocabularios únicos, uno para palabras que muestran sentimientos positivos y otro para los negativos, donde cada uno incluye una única aparición de cada palabra, pero que como valor tiene el número total de apariciones de dicha palabra en el texto; sería como una especie de bag of words.

A continuación, se explica el funcionamiento general del código implementado para el análisis de sentimientos utilizando vocabularios de palabras únicas (positivas y negativas). Este proceso se desglosa en varios pasos clave:

Primero, se crean dos conjuntos vacíos para almacenar las palabras asociadas a sentimientos positivos y negativos. Los conjuntos son especialmente útiles en este caso porque no permiten duplicados, asegurando que cada palabra sólo aparezca una vez en cada vocabulario.

En el siguiente paso, el código recorre todas las reseñas etiquetadas como "positivas" en el conjunto de datos. Para cada reseña, el texto se convierte a minúsculas y se divide en palabras. Estas palabras se añaden al vocabulario positivo. De manera similar, el código recorre las reseñas etiquetadas como "negativas" y realiza el mismo proceso de conversión y tokenización para generar el vocabulario negativo.

Una vez creados ambos vocabularios, el código implementa una función que clasifica un texto según el número de coincidencias de palabras con los vocabularios positivo y negativo. Para cada texto, se convierte el contenido a minúsculas y se divide en palabras. Luego, calculamos el número de palabras coincidentes con los vocabularios de sentimientos positivos y negativos. Si el número de coincidencias con el vocabulario positivo es mayor que con el negativo, el texto se clasifica como positivo; de lo contrario, se clasifica como negativo.

Seguidamente, el conjunto de datos se divide en el conjunto de entrenamiento y el conjunto de pruebas.

Finalmente, se aplica la función de clasificación a cada uno de los textos del conjunto de prueba, generando las predicciones. Para evaluar la efectividad del modelo, se calcula la *F1 Score* ponderada.

- **Método basado en Word embeddings**

En esta ocasión, se propone un método basado en *Word embeddings*, que es una técnica que ya utilizamos en apartados anteriores, por lo que la lógica que seguiremos es la misma que en otras ocasiones.

Implementamos un modelo de análisis de sentimientos que utiliza el modelo preentrenado *DistilBERT* junto con un clasificador *Random Forest*. El flujo de trabajo se desarrolla en varios pasos.

El modelo *DistilBERT* y su tokenizador son cargados desde la librería *transformers*. La función *text_to_embedding* toma un texto, lo “tokeniza”, lo pasa por el modelo para obtener sus representaciones, y devuelve un vector denso que representa el texto.

A continuación, se selecciona una muestra aleatoria del conjunto de datos para reducir el tiempo de procesamiento, utilizando solo el 5% de los datos. Este subconjunto se divide en dos partes: un conjunto de entrenamiento y otro de prueba, con un 30% de los datos reservados para las pruebas.

Luego, se generan los embeddings de los textos tanto para el conjunto de entrenamiento como para el de prueba utilizando la función *text_to_embedding*. Los embeddings resultantes son almacenados en arrays de NumPy.

Con los embeddings generados, se entrena un clasificador *Random Forest*, que se ajusta a los datos de entrenamiento. El clasificador utiliza los embeddings como características y las etiquetas de sentimiento como objetivos.

Una vez entrenado, el modelo hace predicciones sobre el conjunto de prueba, y su rendimiento se evalúa mediante el *F1 Score*.

Este enfoque combina la capacidad de un modelo avanzado como *DistilBERT* para representar texto con la simplicidad y efectividad de un clasificador *Random Forest*, proporcionando una solución robusta para la clasificación de sentimientos.

Conclusiones

Haciendo una valoración general de la efectividad y precisión de los modelos, observamos que claramente con el que se obtienen peores resultados es con el que está basado en *Lexicons*; probablemente, esto se deba a que resulta casi imposible abarcar la mayoría de las palabras presentes en los *posts*, por lo que quizás no se tenga etiqueta para algunas de ellas. Por otro lado, tanto el método basado en palabras únicas como en *Word embeddings* muestran comportamientos similares y bastante precisos. Quizás el método basado en vocabularios únicos sea el mejor para este caso concreto, juzgándolo por un equilibrio entre tiempo de ejecución y precisión, pues es cierto que obtienen grandes resultados, a pesar de ser un proceso relativamente sencillo. No obstante, creemos que, muy bien entrenado y a largo plazo, se podría ajustar como el mejor modelo el basado en *Word embeddings*, pues es el que utiliza también esa componente de Machine Learning, que nos abre aún más el abanico de posibilidades. A pesar de ello, no deja de ser el modelo más costoso computacionalmente, por ello, únicamente, lo probamos con una muestra del total del conjunto de datos.

Es importante mencionar, que algunas explicaciones relevantes para el entendimiento de los códigos también se encuentran en los comentarios de los mismos.